Python

Raccolta Completa: 9 Lezioni ed Esercizi

Documento consolidato con testo integrale, tabelle, formule e tutti gli schemi visivi.
Formattato appositamente per l'ingestione e lo studio su Google NotebookLM.

✨ Fonte Unica per Google NotebookLM

> ⚡ **In Sintesi (A Colpo d'Occhio)**
>
> - **Natura**: Linguaggio ad alto livello, multi-paradigma (imperativo, OOP, funzionale), interpretato tramite **bytecode su PVM**.
> - **Filosofia (*Zen di Python*)**: *"Readability counts"* — la leggibilità e l'espressività del codice hanno la priorità assoluta su tutto.
> - **Tipizzazione Dinamica**: il tipo appartiene all'**oggetto** in memoria Heap, non alla variabile (niente `int x;`).
> - **Tipizzazione Forte**: vietate conversioni implicite incoerenti (es. `"ciao" + 5` solleva **`TypeError`**, a differenza di JS).
> - **Memoria Automatica**: niente puntatori manuali né `delete`; liberazione automatica gestita da **Reference Counting** e **Garbage Collector**.
> - **Esecuzione**: Compilazione trasparente in file `.pyc` (memorizzati in `__pycache__`) eseguiti dalla **Python Virtual Machine (PVM)**.

---

1. Cos'è Python e la sua Filosofia

Ideato nel **1991** dall'informatico olandese **Guido van Rossum** e battezzato in onore del gruppo comico britannico *Monty Python*, Python è oggi il linguaggio di riferimento mondiale per intelligenza artificiale, data science, automazione, scripting di sistema e sviluppo backend web.

A differenza del C++ (progettato per dare il massimo controllo sull'hardware e la massima velocità d'esecuzione), Python è progettato per massimizzare la **produttività dello sviluppatore** e la **chiarezza del codice**.

> 📜 **Lo Zen di Python (PEP 20)**
>
> Aprendo una shell Python e digitando `import this`, viene stampato il manifesto filosofico del linguaggio scritto da Tim Peters:
>
> - *Beautiful is better than ugly.* (Bello è meglio di brutto)
> - *Explicit is better than implicit.* (Esplicito è meglio di implicito)
> - *Simple is better than complex.* (Semplice è meglio di complesso)

> - *Readability counts.* (La leggibilità conta)
> - *There should be one— and preferably only one —obvious way to do it.* (Dovrebbe esserci un solo modo evidente per fare una cosa)

---

2. Come Funziona l'Esecuzione: Il Modello Bytecode e PVM

Spesso si definisce Python come un linguaggio *"interpretato"*, ma tecnicamente è un **linguaggio a esecuzione virtuale su Bytecode** (in modo del tutto analogo a Java o C#).

Quando lanciamo un comando come `python main.py`, avvengono due passaggi trasparenti:

```
flowchart TD
    A["Codice Sorgente\n(script.py)"] -->|Compilatore CPython| B["Bytecode Portabile
    B -->|Caricato nella| C["Python Virtual Machine (PVM)\n(Interprete di loop istru
    C -->|Traduzione istantanea| D["Codice Macchina CPU\n(Hardware Host)"]

    style A fill:#2b5b84,stroke:#306998,color:#fff
    style B fill:#ffe873,stroke:#ffd43b,color:#000
    style C fill:#4b8bbe,stroke:#306998,color:#fff
    style D fill:#306998,stroke:#1e415e,color:#fff
```

1. **Compilazione in Bytecode**:
   - Il compilatore Python analizza la sintassi e traduce il testo in istruzioni binarie ad alto livello chiamate **Bytecode**.
   - Per non ricompilare a ogni avvio, il bytecode viene salvato su disco nella cartella `__pycache__` con estensione `.pyc`. Se il file `.py` non viene modificato, all'avvio successivo Python userà direttamente il file compilato.
2. **Esecuzione tramite PVM (Python Virtual Machine)**:
   - La PVM è un motore software che legge sequenzialmente le istruzioni di bytecode e le esegue traducendole in chiamate alle primitive del sistema operativo sottostante.

> 💡 **Implementazioni di Python**

> L'implementazione ufficiale e standard di riferimento è scritta in linguaggio C ed è denominata **CPython**. Ne esistono altre specializzate: **PyPy** (con compilatore Just-In-Time per prestazioni estreme), **Jython** (per integrarsi nell'ecosistema Java/JVM) e **MicroPython** (per microcontrollori e IoT).

---

3. Il Sistema dei Tipi: Dinamico e Forte

Il sistema dei tipi di Python è definito da due caratteristiche precise che spesso i programmatori alle prime armi confondono:

```
↔flowchart LR
    subgraph TIPI ["Caratteristiche del Sistema di Tipi"]
        TD["Tipizzazione Dinamica\nIl tipo è associato al DATO/OGGETTO,\nnon al nome
        TF["Tipizzazione Forte\nNessuna conversione implicita 'magica'.\nOperazioni
    end
    style TD fill:#e1f5fe,stroke:#03a9f4,color:#01579b
    style TF fill:#e8f5e9,stroke:#4caf50,color:#1b5e20
```

A. Tipizzazione Dinamica (Dynamic Typing)

In C++ dichiariamo la scatola e il suo tipo (`int x = 10;`). In Python dichiariamo solo l'etichetta associata all'oggetto:

```
x = 10          # x punta a un oggetto intero
print(type(x))  # <class 'int'>

x = "Antigravity"  # La stessa variabile x ora punta a una stringa!
print(type(x))  # <class 'str'>
```

B. Tipizzazione Forte (Strong Typing)

In linguaggi a tipizzazione debole (come JavaScript o PHP), sommare un numero e una stringa produce conversioni automatiche bizzarre (`"5" + 2` diventa `"52"`). Python **rifiuta**

categoricamente questo comportamento:

```
numero = 10
testo = "20"

# print(numero + testo)  <-- ERRORE FATALE!
# TypeError: unsupported operand type(s) for +: 'int' and 'str'

# Conversione esplicita obbligatoria (Casting):
print(numero + int(testo))   # 30 (somma algebrica)
print(str(numero) + testo)   # "1020" (concatenazione di stringhe)
```

> 🚫 **Attenzione al Typing Dinamico nei Grandi Progetti**
>
> Poiché non serve dichiarare i tipi, è facile commettere errori di svista passando per sbaglio un testo dove una funzione si aspetta un numero. Dal Python 3.5 in poi è buona pratica usare i **Type Hints** (es. `def calcola(x: int) -> float:`).

---

4. Gestione della Memoria e Garbage Collection

In C++ lo sviluppatore deve gestire manualmente la vita degli oggetti con allocazioni su Stack o con `new` e `delete` su Heap. In Python:

1. **Ogni singolo dato vive nello Heap**: numeri, stringhe, liste, funzioni e classi sono tutti **oggetti**.
2. **Reference Counting (Conteggio dei Riferimenti)**:
   - Ogni oggetto memorizza un contatore interno (`ob_refcnt`).
   - Quando crei una variabile che punta all'oggetto, il contatore sale di 1.
   - Quando una variabile esce dallo scope o viene riassegnata, il contatore scende di 1.
   - **Appena il contatore raggiunge zero**, la memoria dell'oggetto viene deallocata all'istante!
3. **Garbage Collector per Riferimenti Circolari**:
   - Per gestire casi complessi in cui l'oggetto A punta all'oggetto B e viceversa (riferimento ciclico), un modulo automatico di Garbage Collection in background scansiona la

memoria ed elimina le isole isolate di oggetti non più raggiungibili dal programma.

---

5. Tabella Comparativa: C++ vs Python

Parametro	C++	Python
Paradigma	Multi-paradigma orientato alle prestazioni	Multi-paradigma orientato all'espressività
Traduzione del codice	**Compilazione AOT** (*Ahead-Of-Time*) direttamente in codice macchina nativo	**Bytecode** intermedio eseguito da Macchina Virtuale (**PVM**)
Dichiarazione Tipi	**Statica**: obbligatoria (`int x = 5;`)	**Dinamica**: dedotta a runtime (`x = 5`)
Controllo Tipi	Rigido a tempo di compilazione	Rigido a tempo di esecuzione (**Strong Typing**)
Gestione Memoria	Manuale (`new` / `delete` , puntatori, RAII)	Automatica (**Reference Counting** + **Garbage Collector**)
Sintassi e Blocchi	Parentesi graffe `{}` e punto e virgola `;`	**Indentazione obbligatoria** a 4 spazi e a-capo
Velocità di Esecuzione	Estrema (vicina al metallo dell'hardware)	Più lenta (compensata da librerie interne scritte in C/C++)
Velocità di Sviluppo	Medio-bassa (più codice cerimoniale da scrivere)	Altissima (prototipazione immediata in poche righe)

---

6. Domande d'Esame e Concetti Fondamentali

> ❓ **Domande Tipiche di Verifica**

1. *Cosa significa che Python è a tipizzazione dinamica ma forte?*
   Risposta: Dinamica perché le variabili non hanno un tipo rigido ma sono etichette che possono puntare a dati di qualsiasi tipo; Forte perché Python non esegue conversioni di tipo implicite e non permette operazioni tra tipi non compatibili.
2. *A cosa servono i file* `.pyc` *dentro la cartella* `__pycache__` *?*
   Risposta: Contengono il bytecode precompilato dello script. Evitano di dover rianalizzare la grammatica del codice ad ogni esecuzione, velocizzando l'avvio.
3. *Come fa Python a sapere quando cancellare un dato dalla RAM?*
   Risposta: Tramite il Reference Counting. Ogni dato tiene traccia di quante variabili lo stanno puntando; quando il contatore arriva a zero, la memoria viene deallocata all'istante.

---

7. Codice di Verifica dell'Ambiente

🧪 Programma Completo: Ispezione dell'Ambiente Python

Copia ed esegui questo codice per visualizzare le caratteristiche della tua installazione:

```
import sys
import platform

def report_sistema():
    print("=" * 45)
    print("  ANALISI AMBIENTE PYTHON")
    print("=" * 45)
    print(f"Versione Python      : {platform.python_version()}")
    print(f"Implementazione      : {platform.python_implementation()}
    print(f"Sistema Operativo    : {platform.system()} {platform.rele
    print(f"Architettura Hardware: {platform.machine()}")
    print(f"Percorso Eseguibile  : {sys.executable}")
    print("=" * 45)

    # Verifica reference counting di un intero
    numero = 42000
    print(f"ID Oggetto in memoria: {hex(id(numero))}")
    print(f"Numero di riferimenti: {sys.getrefcount(numero) - 1}")
    print("=" * 45)

if __name__ == "__main__":
    report_sistema()
```

> **In Sintesi (A Colpo d'Occhio)**
>
> - **Variabile come Etichetta (Binding)**: una variabile non è una scatola fissa di memoria, ma un **nome che punta a un oggetto** nello Heap.
> - **Tipi Base Immutabili**: `int` (interi a precisione infinita, niente overflow!), `float` (64-bit IEEE 754), `str` (testo Unicode immutabile), `bool` (`True` / `False`), `None` (assenza di valore).
> - **Type Annotations**: `x: int = 10` documenta il tipo atteso senza bloccare l'esecuzione a runtime.
> - **Input/Output**: `print(f"Risultato: {x:.2f}", end="\n")` e `input()` .
> - **Trap di `input()`** : `input()` legge **SEMPRE una stringa**; per fare calcoli numerici è obbligatorio il casting esplicito `int(input())` .
> - **Operatori Chiave**: `/` (divisione decimale reale), `//` (divisione intera con troncamento), `%` (modulo), `**` (elevamento a potenza).

---

1. Il Modello di Variabile: Scatola (C++) vs Etichetta (Python)

In linguaggi come il C++, una variabile è una cella fissa di memoria RAM con un tipo prefissato: In **Python**, invece, **tutto è un oggetto** allocato nello Heap. Una variabile è semplicemente un'**etichetta (binding)** legata con un riferimento a un oggetto.

```
flowchart LR
    subgraph STACK ["Nomi (Scope / Simboli)"]
        A["eta"]
        B["anni"]
    end

    subgraph HEAP ["Oggetti in Memoria Heap"]
        OBJ["Oggetto Intero\nTipo: int\nValore: 18\nID: 0x7fa2..."]
    end

    A -->|Riferimento| OBJ
    B -->|Riferimento| OBJ

    style A fill:#4b8bbe,stroke:#306998,color:#fff
    style B fill:#4b8bbe,stroke:#306998,color:#fff
    style OBJ fill:#ffe873,stroke:#ffd43b,color:#000
```

Se scriviamo:

```
eta = 18
anni = eta
```

Non stiamo duplicando il numero `18` in due celle diverse: abbiamo semplicemente creato due etichette (`eta` e `anni`) che puntano allo **stesso identico oggetto** in memoria!

---

2. I Tipi di Dato Primitivi

Tutti i tipi primitivi standard in Python sono **immutabili**: una volta creato l'oggetto, il suo valore interno non può essere modificato. Se assegni un nuovo valore, viene creato un nuovo oggetto nello Heap.

Tipo	Descrizione	Esempio	Note & Differenze con C++
`int`	Numero intero con segno	`x = 42`	**Precisione arbitraria infinita**: non va mai in overflow (a differenza di `int32` o `int64` in C++).
`float`	Numero in virgola mobile	`pi = 3.14159`	Equivale al `double` del C++ (64 bit in virgola mobile standard IEEE 754).
`str`	Sequenza di caratteri Unicode	`msg = "Ciao!"`	Racchiuso tra apici singoli `'...'` o doppi `"..."`. Immutabile.
`bool`	Valore di verità booleano	`attivo = True`	Solo due valori possibili: `True` o `False` (con la prima lettera maiuscola!).
`NoneType`	Assenza di valore o valore nullo	`risultato = None`	Il valore speciale `None` corrisponde al concetto di `null` o `nullptr`.

La Meraviglia degli Interi senza Overflow

In C++, un intero a 32 bit oltrepassa i 2.147.483.647 e genera un overflow silenzioso diventando negativo. In Python la memoria per gli interi cresce dinamicamente:

```
valore_gigante = 2 ** 100
print(valore_gigante)
# Stampa: 1267650600228229401496703205376 (Calcolo esatto perfetto!)
```

---

3. Ispezione dei Tipi: `type()` vs `isinstance()`

Per verificare il tipo effettivo a cui punta una variabile possiamo usare due funzioni:

```
x = 42

# 1. Verifica con type() (confronto stretto)
print(type(x))             # <class 'int'>
print(type(x) is int)      # True

# 2. Verifica con isinstance() (CONSIGLIATA: rispetta l'ereditarietà OOP)
print(isinstance(x, int))   # True
print(isinstance(x, (int, float))) # True (verifica multipla)
```

> 🎯 **Regola d'Oro**
>
> Nei controlli di validazione dei parametri all'interno dei tuoi programmi, preferisci sempre **`isinstance(dato, Tipo)`** rispetto a `type(dato) == Tipo`, poiché `isinstance` supporta l'ereditarietà delle classi.

---

4. Type Hints (Annotazioni di Tipo Moderne)

Introdotte con la **PEP 484**, le annotazioni di tipo rendono il codice Python auto-esplicativo e permettono agli strumenti di analisi (come i linter degli IDE) di individuare bug prima dell'esecuzione:

```
# Sintassi: nome_variabile: Tipo = valore_iniziale
studente: str = "Alessandro"
eta: int = 18
media_voti: float = 8.75
promosso: bool = True
```

> **NOTE**
>
> Le annotazioni di tipo **non influenzano le prestazioni a runtime** e Python non blocca l'esecuzione se assegni un tipo diverso. Servono per la documentazione, la leggibilità e

> l'autocompletamento dell'editor.

---

5. Input e Output da Terminale

A. Output Avanzato con `print()` e le f-strings

La funzione `print()` accetta parametri speciali come `sep` (separatore tra gli argomenti) ed `end` (carattere finale, di default `\n`):

```
print("Mela", "Pera", "Banana", sep=" - ")  # Mela - Pera - Banana
print("Attendere", end="...")
print(" Fatto!")  # Stampa sulla stessa riga: Attendere... Fatto!
```

Il modo più moderno, efficiente e pulito per formattare stringhe è l'uso delle **f-strings** (prefisso `f"`):

```
nome = "Luca"
saldo = 1250.456

# Interpolazione diretta ed espressioni matematiche
print(f"Benvenuto {nome.upper()}, il tuo saldo è €{saldo:.2f}!")
# Output: Benvenuto LUCA, il tuo saldo è €1250.46!
```

B. Input da Tastiera con `input()`

La funzione `input(messaggio)` sospende il programma in attesa che l'utente scriva e prema Invio.

> 🚫 **La Trappola più Frequente di `input()`**
>
> `input()` restituisce **SEMPRE e SOLO una stringa (`str`)**, anche se l'utente digita delle cifre!

```
a = input("Inserisci primo numero: ")  # Utente digita: 5
b = input("Inserisci secondo numero: ") # Utente digita: 3
print(a + b) # Stampa: "53" (concatenazione testuale!)
```

Per eseguire somme matematiche, è **obbligatorio effettuare il casting**:

```
num1 = int(input("Inserisci primo numero: "))
num2 = int(input("Inserisci secondo numero: "))
print(num1 + num2) # Stampa: 8 (somma numerica corretta)
```

---

6. Operatori Aritmetici

Python possiede operatori matematici dedicati che evitano i classici trabocchetti di linguaggi a tipizzazione debole:

Operatore	Significato	Esempio	Risultato	Note
`+`	Somma algebrica o concatenazione	`10 + 5`	`15`	Se usato tra stringhe fa concatenazione (`"a" + "b"` → `"ab"`).
`-`	Sottrazione	`10 - 3`	`7`	
`*`	Moltiplicazione o ripetizione	`4 * 3`	`12`	Se usato tra stringa e int ripete il testo (`"-" * 5` → `"-----"`).
`/`	**Divisione reale (decimale)**	`5 / 2`	**`2.5`**	**Restituisce sempre un `float`**, anche se i due operandi sono interi!
`//`	**Divisione intera (Floor Div)**	`5 // 2`	**`2`**	Tronca all'intero inferiore più vicino.
`%`	**Modulo (Resto)**	`5 % 2`	**`1`**	Resto della divisione euclidea intera.
`**`	**Elevamento a potenza**	`2 ** 8`	**`256`**	28 = 256, non serve importare librerie esterne.

Operatori di Assegnazione Composta

Come in C++, possiamo combinare calcolo e riassegnazione:

```
punti = 100
punti += 10    # punti = punti + 10 -> 110
punti //= 2    # divisione intera -> 55
punti **= 2    # elevamento al quadrato -> 3025
```

---

7. Domande d’Esame e Concetti Chiave

❓ Domande di Verifica

1. *Qual è la differenza tra l’operatore `/` e l’operatore `//` in Python?*
   Risposta: `/` calcola la divisione reale restituendo sempre un `float` (es. `7 / 2` produce `3.5`). L’operatore `//` esegue la divisione intera troncata al pavimento (es. `7 // 2` produce `3`).
2. *Se eseguo `a = 10` e poi `b = a`, cosa succede fisicamente in memoria?*
   Risposta: Viene creato un singolo oggetto intero `10` nello Heap; le due variabili `a` e `b` puntano contemporaneamente allo stesso indirizzo di memoria dell’oggetto.
3. *Cosa succede se provi a sommare un numero intero e una stringa numerica (es. `10 + "5"`)?*
   Risposta: Python solleva un’eccezione `TypeError` perché è un linguaggio a tipizzazione forte e rifiuta conversioni implicite ambigue.

---

8. Programma Esecutivo Completo

💉 Programma Completo: Calcolatore di Spesa con Scontrino

```
def calcolatore_spesa():
    print("--- GESTIONE CASSA AUTOMATICA ---")

    prodotto: str = input("Nome del prodotto: ")
    prezzo_unitario: float = float(input("Prezzo unitario (€): "))
    quantita: int = int(input("Quantità acquistata: "))
    percentuale_iva: float = 22.0

    subtotale: float = prezzo_unitario * quantita
    importo_iva: float = subtotale * (percentuale_iva / 100)
    totale: float = subtotale + importo_iva

    # Stampa scontrino con formattazione tabellare f-string
    print("\n" + "=" * 40)
    print(f"{'SCONTRINO FISCALE':^40}")
    print("=" * 40)
    print(f"Articolo:        {prodotto.strip().capitalize()}")
    print(f"Prezzo x Qta:    €{prezzo_unitario:.2f} x {quantita}")
    print(f"Subtotale netto: €{subtotale:.2f}")
    print(f"IVA ({percentuale_iva:.0f}%):       €{importo_iva:.2f}")
    print("-" * 40)
    print(f"TOTALE DA PAGARE: €{totale:.2f}")
    print("=" * 40)

if __name__ == "__main__":
    calcolatore_spesa()
```

> **In Sintesi (A Colpo d'Occhio)**
>
> - **Indentazione Obbligatoria**: in Python i blocchi di codice non usano le parentesi graffe `{}`, ma **esattamente 4 spazi di rientro**.
> - **Selezione**: `if`, `elif` (contrazione di *else if*), `else`.
> - **Operatori Logici Parola**: `and`, `or`, `not` (sostituiscono i simboli `&&`, `||`, `!` del C++).
> - **Chained Comparisons**: confronti a catena naturali (es. `18 <= voto <= 30`).
> - **Operatore Ternario**: `stato = "Promosso" if voto >= 18 else "Bocciato"`.
> - **Pattern Matching Moderno**: costrutto `match - case` (da Python 3.10+) senza bisogno di `break` anti-fallthrough.
> - **Iterazione Pythonica**: `for ... in range(n)`, `enumerate()` per indice + elemento contemporanei, e clausola speciale `else` nei cicli.

---

1. L'Indentazione Significativa: La Regola d'Oro di Python

In C++ le parentesi graffe `{}` delimitano i blocchi di codice e l'indentazione è solo una convenzione estetica. In **Python l'indentazione è parte integrante della grammatica del linguaggio**:

```
flowchart TD
    A["Istruzione Esterna"] --> B{"Condizione?"}
    B -- True --> C["    Istruzione Indentata 1\n    Istruzione Indentata 2"]
    B -- False --> D["    Istruzione Alternativa"]
    C --> E["Istruzione Successiva (Ritorno a indentazione 0)"]
    D --> E

    style C fill:#d4edda,stroke:#28a745,color:#155724
    style D fill:#f8d7da,stroke:#dc3545,color:#721c24
```

> **Errore Comune: `IndentationError` e `TabError`**

> Non mescolare mai **Tabulazioni (Tab)** e **Spazi**. Lo standard ufficiale PEP 8 impone l'uso di **4 spazi per ciascun livello di indentazione**. Se l'allineamento è errato, il programma non parte nemmeno!

---

2. Selezione Condizionale: `if`, `elif`, `else`

La sintassi prevede la parola chiave seguita dalla condizione e dal simbolo di due punti `:`:

```
eta = 17

if eta >= 18:
    print("Sei maggiorenne: puoi votare.")
elif eta >= 14:
    print("Sei un teenager: puoi guidare un ciclomotore.")
else:
    print("Sei minorenne.")
```

Operatori Logici e Relazionali

A differenza dei simboli matematici crudi del C++ (`&&`, `||`, `!`), Python usa parole inglesi pulite:

- **`and`** : vero se entrambe le condizioni sono vere (con cortocircuito: se la prima è falsa, la seconda non viene nemmeno valutata).
- **`or`** : vero se almeno una condizione è vera.
- **`not`** : negazione logica (`not True` → `False`).
- **`in`** / **`not in`** : verifica immediata di appartenenza ad una sequenza o stringa:

```
if "admin" in ruolo_utente:
    print("Accesso consentito")
```

Confronti a Catena (Chained Comparisons)

In C++ dobbiamo scrivere `voto >= 18 && voto <= 30`. In Python la sintassi è speculare a quella della matematica scolastica:

```
voto = 27
if 18 <= voto <= 30:
    print("Voto valido per il superamento dell'esame.")
```

Operatore Ternario (Espressione Condizionale Monoriga)

Consente di assegnare un valore in base a una condizione in una sola riga leggibile:

```
voto = 15
esito = "Superato" if voto >= 18 else "Non superato"
print(esito)  # Non superato
```

---

3. Selezione Multipla Moderna: `match - case`

Introdotto in **Python 3.10** (PEP 634), il costrutto `match - case` supera lo `switch-case` del C++ grazie al **Pattern Matching Strutturale**:

1. Non richiede la parola chiave `break` (nessun rischio di *fallthrough* accidentale).
2. Usa il carattere di sottolineatura `_` come caso predefinito (*default wildcard*).
3. Permette di unire più casi con la barra verticale `|`.

```
comando = "SALVA"

match comando.upper():
    case "AVVIA" | "START":
        print("Sistema avviato.")
    case "FERMA" | "STOP":
        print("Arresto del sistema.")
    case "SALVA":
        print("Dati salvati su disco.")
    case _:
        print("Comando non riconosciuto!")
```

---

4. Cicli Iterativi: `while` e `for`

A. Ciclo `while` e la Clausola `while ... else`

Il ciclo `while` esegue il blocco finché la condizione resta vera.

```
conteggio = 3
while conteggio > 0:
    print(f"Conto alla rovescia: {conteggio}")
    conteggio -= 1
else:
    print("Lancio avvenuto con successo!")
```

> 💡 **La Clausola `else` nei Cicli**
>
> Il blocco `else` abbinato a un ciclo `while` (o `for`) viene eseguito **solo se il ciclo è giunto al termine naturale** (senza essere interrotto bruscamente da un'istruzione `break`). È l'ideale per le ricerche!

B. Ciclo `for` e la Funzione `range()`

In Python il ciclo `for` non è un contatore manuale con indici, ma un **iteratore che attraversa una sequenza**:

```
# Sintassi di range: range(inizio, fine_esclusa, passo)
# Intervallo semi-aperto [0, 5) -> genera 0, 1, 2, 3, 4
for i in range(5):
    print(i, end=" ")
# Output: 0 1 2 3 4
```

Esempi di parametri con `range`:

- `range(1, 6)` → genera 1, 2, 3, 4, 5.
- `range(10, 0, -2)` → conto alla rovescia con passo −2: 10, 8, 6, 4, 2.

---

5. Strumenti Iterativi Idiomatici: `enumerate()` e `zip()`

> 🎯 **Regola d'Oro Pythonica**
>
> Non usare mai variabili contatore manuali prima del ciclo (`i = 0; while ... i += 1`). Python offre funzioni native integrate progettate appositamente.

1. `enumerate()` : Indice e Valore Insieme

Quando scorri una lista e hai bisogno sia dell'indice numerico sia dell'elemento:

```
linguaggi = ["Python", "C++", "JavaScript", "Rust"]

for indice, nome in enumerate(linguaggi, start=1):
    print(f"{indice}. {nome}")
# Output:
# 1. Python
# 2. C++
# 3. JavaScript
# 4. Rust
```

2. `zip()` : Scorrere Più Liste in Parallelo

Evita di accedere tramite indici a vettori paralleli:

```
nomi = ["Mario", "Luigi", "Peach"]
punti = [1200, 950, 1500]

for n, p in zip(nomi, punti):
    print(f"Giocatore: {n} - Punteggio: {p}")
```

---

6. Salto di Flusso: `break`, `continue` e `pass`

- **`break`**: termina immediatamente il ciclo più interno.
- **`continue`**: salta all'iterazione successiva ignorando il codice rimanente nel blocco.
- **`pass`**: istruzione "segnaposto" (*no-operation*). Serve quando la sintassi richiede un blocco di codice ma non si vuole eseguire nulla (es. funzioni o classi ancora da implementare).

```
for num in range(1, 10):
    if num % 2 == 0:
        continue  # Salta i numeri pari
    if num == 7:
        print("Trovato il numero 7! Arresto anticipato.")
        break     # Termina il ciclo
    print(f"Numero dispari: {num}")
```

---

7. Domande d'Esame e Concetti Chiave

> ❓ **Domande di Verifica**
>
> 1. *Cosa succede se in un programma Python si usa un'indentazione incoerente (es. a volte 2 spazi e a volte 4)?*
>   **Risposta**: L'interprete rifiuta di eseguire il programma e genera un'eccezione `IndentationError` prima del runtime.
> 2. *Come funziona la clausola `else` applicata a un ciclo `for`?*
>   **Risposta**: Il blocco `else` viene eseguito se il ciclo termina normalmente dopo aver consumato tutti gli elementi. Se il ciclo viene interrotto prima con un'istruzione `break`, il blocco `else` viene saltato.
> 3. *Quali vantaggi offre il `match-case` rispetto allo `switch` del C++?*
>   **Risposta**: Non necessita di `break` a fine di ogni caso, non soffre di fallthrough accidentale, consente di verificare tipi complessi, sequenze e applicare filtri di guardia (`if`).

---

8. Programma Esecutivo Completo

🧪 Programma Completo: Gioco "Indovina il Numero Segreto"

```
import random

def gioca_indovinello():
    numero_segreto: int = random.randint(1, 50)
    tentativi_massimi: int = 5

    print("=== BENVENUTO NEL GIOCO DELL'INDOVINELLO ===")
    print(f"Ho scelto un numero tra 1 e 50. Hai a disposizione {tenta

    for tentativo in range(1, tentativi_massimi + 1):
        scelta_utente = input(f"Tentativo {tentativo}/{tentativi_mass

        if scelta_utente.lower().strip() == "esci":
            print("Partita interrotta dall'utente.")
            break

        if not scelta_utente.isdigit():
            print("⚠️ Inserisci un numero intero valido!")
            continue

        valore = int(scelta_utente)

        if valore == numero_segreto:
            print(f"🎉 COMPLIMENTI! Hai indovinato il numero {numero_
            break
        elif valore < numero_segreto:
            print("📈 Troppo basso! Prova un numero più alto.")
        else:
            print("📉 Troppo alto! Prova un numero più basso.")
    else:
        # Viene eseguito SOLO se i tentativi finiscono senza break
        print(f"\n❌ Tentativi esauriti! Il numero segreto era: {nume

if __name__ == "__main__":
    gioca_indovinello()
```

> **In Sintesi (A Colpo d'Occhio)**
>
> - **Le 4 Strutture Native**:
>   - `list`: ordinata, mutabile, eterogenea (`[1, 2, "tre"]`).
>   - `tuple`: ordinata, immutabile, protetta da scritture accidentali (`(10, 20)`).
>   - `set`: non ordinata, elementi univoci (elimina duplicati all'istante), ricerca in O(1).
>   - `dict`: coppie chiave-valore ad accesso istantaneo hash table O(1) (`{"nome": "Luca"}`).
> - **Indici Negativi**: `lista[-1]` restituisce l'ultimo elemento, `lista[-2]` il penultimo.
> - **Slicing**: estrazione fette con `lista[inizio:fine_esclusa:passo]`; per invertire una lista: `lista[::-1]`.
> - **List Comprehension**: sintassi compatta ad alta velocità `[x**2 for x in lista if x > 0]` .

---

1. Panoramica delle 4 Strutture Dati Native

In C++ per collezionare dati si usano gli array statici `int v[100]` o i contenitori della STL come `std::vector` e `std::map`. Python integra nel suo nucleo **4 collezioni native ad altissimo livello**:

```
flowchart TD
    COL["Collezioni di Dati in Python"]
    COL --> L["1. LISTA (list)\n[1, 2, 3]\nOrdinata, Mutabile, Ammette Duplicati"]
    COL --> T["2. TUPLA (tuple)\n(1, 2, 3)\nOrdinata, Immutabile, Ammette Duplicati"
    COL --> S["3. INSIEME (set)\n{1, 2, 3}\nNon Ordinato, Mutabile, NO Duplicati"]
    COL --> D["4. DIZIONARIO (dict)\n{'a': 1, 'b': 2}\nChiave-Valore, Ricerca Istant

    style L fill:#d4edda,stroke:#28a745,color:#155724
    style T fill:#cce5ff,stroke:#007bff,color:#004085
    style S fill:#fff3cd,stroke:#ffc107,color:#856404
    style D fill:#f8d7da,stroke:#dc3545,color:#721c24
```

Tipo	Sintassi	Ordinata?	Mutabile?	Duplicati?	Caso d'Uso Principale
`list`	`[a, b, c]`	**Sì**	**Sì**	**Sì**	Sequenze dinamiche di elementi generici.
`tuple`	`(a, b, c)`	**Sì**	**No**	**Sì**	Record fissi, coordinate geografiche, chiavi immutabili.
`set`	`{a, b, c}`	**No**	**Sì**	**No**	Test di appartenenza ultra-veloce O(1), eliminazione duplicati.
`dict`	`{k: v}`	**Sì** (da Py 3.7+)	**Sì**	Solo valori	Tabelle di associazione, database in memoria, record JSON.

---

2. Le Liste (`list`): Array Dinamico

Una lista è una sequenza flessibile in cui gli elementi possono essere di qualunque tipo (anche eterogenei):

```
numeri = [10, 20, 30, 40, 50]
misto = [1, "Python", 3.14, True]
vuota = []
```

Indicizzazione Zero-Based e Indici Negativi

In Python possiamo contare anche partendo dalla fine della lista con il segno meno:

```
voti = [24, 28, 30, 18, 27]

print(voti[0])   # 24 (primo elemento)
print(voti[-1])  # 27 (ultimo elemento: niente più 'voti[len(voti) - 1]' c
print(voti[-2])  # 18 (penultimo elemento)
```

Lo Slicing: Estrazione di Sottosequenze

Lo slicing estrae una porzione della lista generando una nuova lista con la sintassi `lista[start:stop:step]`:

```
lettere = ['A', 'B', 'C', 'D', 'E', 'F']

print(lettere[1:4])   # ['B', 'C', 'D'] (da indice 1 a indice 4 ESCLUSO)
print(lettere[:3])    # ['A', 'B', 'C'] (dall'inizio fino a indice 3 esclus
print(lettere[3:])    # ['D', 'E', 'F'] (da indice 3 fino alla fine)
print(lettere[::2])   # ['A', 'C', 'E'] (tutti con passo 2)
print(lettere[::-1])  # ['F', 'E', 'D', 'C', 'B', 'A'] (LISTA INVERTITA!)
```

Metodi Fondamentali delle Liste

```
valori = [10, 20, 30]

# Inserimento
valori.append(40)          # Inserisce in coda -> [10, 20, 30, 40]
valori.insert(1, 15)       # Inserisce all'indice 1 -> [10, 15, 20, 30, 40]
valori.extend([50, 60])    # Aggiunge più elementi -> [..., 50, 60]

# Rimozione
elemento = valori.pop()    # Rimuove e restituisce l'ultimo (60)
valori.remove(15)          # Cerca ed elimina il primo valore uguale a 15
del valori[0]              # Elimina per posizione (indice 0)

# Statistiche integrate
print(len(valori))         # Lunghezza
print(min(valori))         # Valore minimo
print(max(valori))         # Valore massimo
print(sum(valori))         # Somma di tutti gli elementi
```

---

3. Matrici in Python (Liste di Liste)

Una matrice è rappresentata come una lista contenente altre liste:

```
matrice = [
    [1, 2, 3],
    [4, 5, 6],
    [7, 8, 9]
]

# Accesso: matrice[riga][colonna]
print(matrice[1][2]) # Stampa 6 (riga 1, colonna 2)
```

> 🚫 **Trappola di Inizializzazione delle Matrici**
>
> **NON inizializzare mai una matrice in questo modo**:

```
righe, colonne = 3, 3
errata = [[0] * colonne] * righe # GRAVISSIMO ERRORE!
```

Questo comando crea 3 righe che puntano **tutte alla medesima riga in memoria**: modificando `errata[0][0] = 5`, tutte e 3 le righe vedranno il 5!
Il modo corretto è usare una list comprehension:

```
corretta = [[0 for _ in range(colonne)] for _ in range(righe)]
```

---

4. List Comprehensions: Il Superpotere di Python

Le **List Comprehensions** permettono di generare, trasformare e filtrare liste con una sintassi concisa, elegante e più veloce dei cicli standard:

Risultato = [espressione for elemento in iterabile if condizione]

```
flowchart LR
    A["iterabile:\n[1, 2, 3, 4, 5, 6]"] -->|Filtro if x % 2 == 0| B["Elementi Pari:\
    B -->|Trasformazione x ** 2| C["Nuova Lista:\n[4, 16, 36]"]

    style A fill:#e1f5fe,stroke:#03a9f4,color:#01579b
    style B fill:#fff9c4,stroke:#fbc02d,color:#f57f17
    style C fill:#d4edda,stroke:#28a745,color:#155724
```

Confronto di Sintassi:

```
# Modo Classico (con ciclo for e append):
quadrati = []
for x in range(1, 11):
    if x % 2 == 0:
        quadrati.append(x ** 2)

# Modo Pythonico con List Comprehension:
quadrati = [x ** 2 for x in range(1, 11) if x % 2 == 0]
# Risultato: [4, 16, 36, 64, 100]
```

---

5. Le altre Collezioni: Tuple, Insiemi e Dizionari

A. Le Tuple (`tuple`)

Identiche alle liste, ma racchiuse da parentesi tonde e **immutabili** (impossibile fare `.append()` o cambiare elementi):

```
punto = (10, 25)
# punto[0] = 15  <-- TypeError: 'tuple' object does not support item assig

# Unpacking istantaneo delle tuple:
x, y = punto
print(f"X={x}, Y={y}")
```

B. Gli Insiemi (`set`)

Collezione di elementi unici, racchiusi tra graffe `{}` :

```
numeri_con_duplicati = [1, 2, 2, 3, 4, 4, 4, 5]
unici = set(numeri_con_duplicati)
print(unici)  # {1, 2, 3, 4, 5} -> duplicati eliminati all'istante!

# Operazioni insiemistiche
a = {1, 2, 3}
b = {3, 4, 5}
print(a | b)  # Unione: {1, 2, 3, 4, 5}
print(a & b)  # Intersezione: {3}
```

C. I Dizionari (`dict`)

Mappano una chiave univoca ad un valore associato:

```
studente = {
    "matricola": 45892,
    "nome": "Marco",
    "corso": "Informatica",
    "voti": [28, 30, 27]
}

# Accesso e modifica
print(studente["nome"])       # Marco
studente["media"] = 28.3      # Inserisce nuova coppia chiave-valore

# Accesso sicuro con .get() (non crasha se la chiave non esiste!)
print(studente.get("email", "Non fornita"))  # Restituisce il default senz
```

---

6. Domande d'Esame e Concetti Chiave

> ❓ **Domande di Verifica**
>
> 1. *Qual è il vantaggio computazionale di usare un `set` o un `dict` per cercare un elemento rispetto a una `list` ?*

Risposta: La ricerca in una lista richiede tempo lineare O(n) scorrendo tutti gli elementi. In `set` e `dict`, basati su tabella hash, la verifica di esistenza avviene in tempo costante medio O(1).

2. *Cosa restituisce l'espressione `lista[::-1]`?*
   Risposta: Restituisce una copia superficiale della lista con gli elementi ordinati in senso inverso, grazie al passo −1 dello slicing.
3. *Perché `[[0] * 3] * 3` è una trappola pericolosa?*
   Risposta: Perché duplica per tre volte il riferimento alla medesima riga, anziché allocare tre liste distinte in memoria.

---

7. Programma Esecutivo Completo

🧪 Programma Completo: Analizzatore di Registro Elettronico

```
def analizza_classe():
    studenti = [
        {"nome": "Alice", "voti": [8, 9, 7, 10]},
        {"nome": "Bob", "voti": [5, 6, 6, 4]},
        {"nome": "Carla", "voti": [9, 10, 9, 9]},
        {"nome": "Davide", "voti": [6, 7, 6, 8]}
    ]

    print("=== REPORT RENDIMENTO CLASSE ===")

    # Calcolo medie con List Comprehensions e dizionari
    medie = {s["nome"]: sum(s["voti"]) / len(s["voti"]) for s in stud

    for nome, media in medie.items():
        esito = "PROMOSSO" if media >= 6 else "DA RECUPERARE"
        print(f"Studente: {nome:<10} | Media: {media:4.2f} | Stato: {

    # Filtraggio studenti eccellenti (media >= 8.5)
    eccellenze = [nome for nome, media in medie.items() if media >= 8
    print("\n🌟 Studenti con borsa di studio (media >= 8.5):", eccell

    # Insieme dei voti unici registrati
    tutti_i_voti = [v for s in studenti for v in s["voti"]]
    voti_distinti = sorted(set(tutti_i_voti))
    print("📊 Voti differenti assegnati nella classe:", voti_distinti

if __name__ == "__main__":
    analizza_classe()
```

> ⚡ **In Sintesi (A Colpo d'Occhio)**
>
> - **Dichiarazione**: keyword **`def nome_funzione(parametri):`**; se non c'è `return`, restituisce implicitamente **`None`**.
> - **Argomenti Flessibili**: supporta argomenti posizionali, nominali (*Keyword Arguments* `f(x=10)`), e valori di default.
> - **`*args` e `**kwargs`**: `*args` raccoglie argomenti posizionali extra in una **tupla**; `**kwargs` raccoglie argomenti con nome in un **dizionario**.
> - **Pass-by-Object-Reference**: se passi un oggetto mutabile (es. una lista) e ne modifichi il contenuto, la modifica sarà visibile anche all'esterno.
> - **Scope LEGB**: la ricerca delle variabili avviene nell'ordine **Local $\rightarrow$ Enclosing $\rightarrow$ Global $\rightarrow$ Built-in**.
> - **Lambda Functions**: funzioni anonime monoriga per trasformazioni rapide `lambda x: x * 2`.

---

1. Definizione e Chiamata di Funzione

In Python le funzioni si definiscono con la parola chiave **`def`**. Non è necessario specificare il tipo di ritorno: se la funzione termina senza un'istruzione `return` esplicita, restituirà automaticamente il valore speciale `None`.

```
def calcola_area_rettangolo(base: float, altezza: float) -> float:
    """Calcola e restituisce l'area di un rettangolo (Docstring)."""
    return base * altezza

# Chiamata
area = calcola_area_rettangolo(5.0, 3.0)
print(f"Area: {area}")  # Area: 15.0
```

> 💡 **Docstring (Documentazione Integrata)**

> La stringa su tre apici `"""..."""` posta subito sotto la definizione della funzione è la **Docstring**. Viene letta automaticamente da `help(nome_funzione)` e mostrata dagli IDE durante la scrittura del codice.

---

2. Parametri e Flessibilità degli Argomenti

In C++ gli argomenti devono corrispondere rigidamente all'ordine e al tipo dichiarato. In Python abbiamo una flessibilità straordinaria:

```
↔flowchart TD
    CALL["Chiamata a Funzione"]
    CALL --> POS["Argomenti Posizionali\nsaluta('Luca', 'Roma')"]
    CALL --> KW["Keyword Arguments\nsaluta(citta='Roma', nome='Luca')"]
    CALL --> DEF["Valori di Default\nsaluta('Luca') -> usa valore predefinito"]

    style CALL fill:#2b5b84,stroke:#306998,color:#fff
    style POS fill:#cce5ff,stroke:#007bff,color:#004085
    style KW fill:#d4edda,stroke:#28a745,color:#155724
    style DEF fill:#fff3cd,stroke:#ffc107,color:#856404
```

Parametri con Valori di Default

```python
def connetti_db(host: str, porta: int = 3306, timeout: int = 10):
    print(f"Connessione a {host}:{porta} (timeout={timeout}s)")

connetti_db("localhost")                    # Usa porta 3306 e timeout 10
connetti_db("192.168.1.1", 5432)            # Sovrascrive solo porta
connetti_db("10.0.0.1", timeout=30)         # Usa keyword argument per salta
```

> 🚫 **Errore Critico: Default Mutabili (*The Mutable Default Trap*)**
>
> **NON usare MAI oggetti mutabili (liste, dizionari) come valori di default!**

```
def aggiungi_elemento(item, lista=[]): # ERRORE PESSIMO!
    lista.append(item)
    return lista

print(aggiungi_elemento(1))  # [1]
print(aggiungi_elemento(2))  # [1, 2]  <- La lista è la stessa della
```

Il parametro di default viene creato **una sola volta** quando la funzione viene definita in memoria.
Soluzione Corretta (Pattern Idiomatico):

```
def aggiungi_elemento(item, lista=None):
    if lista is None:
        lista = []  # Crea una nuova lista ad OGNI invocazione
    lista.append(item)
    return lista
```

---

3. Parametri Arbitrari: `*args` e `**kwargs`

Quando non sappiamo a priori quanti argomenti l'utente passerà alla funzione:

`*args` (Argomenti Posizionali Arbitrari)

Raccoglie tutti gli argomenti posizionali extra in una **tupla**:

```
def somma_tutti(*numeri):
    # 'numeri' è una tupla contenente tutti gli argomenti passati
    totale = 0
    for n in numeri:
        totale += n
    return totale

print(somma_tutti(1, 2, 3))          # 6
print(somma_tutti(10, 20, 30, 40))   # 100
```

`**kwargs` (Keyword Arguments Arbitrari)

Raccoglie tutti gli argomenti con nome non previsti in un **dizionario**:

```
def crea_profilo(nome: str, **dettagli):
    # 'dettagli' è un dizionario chiave: valore
    print(f"Profilo di: {nome}")
    for k, v in dettagli.items():
        print(f" - {k.capitalize()}: {v}")

crea_profilo("Alessandro", ruolo="Admin", citta="Milano", livello=4)
```

---

4. Meccanismo di Passaggio: *Pass-by-Object-Reference*

In C++ abbiamo passaggio per valore (`int x`) e per riferimento (`int &x`).
In Python esiste un unico meccanismo detto **Pass-by-Object-Reference** (o *Call-by-Sharing*):

- Alla funzione viene passato il **riferimento all'oggetto originale**.
- Se l'oggetto è **immutabile** (`int` , `float` , `str` , `tuple`), qualsiasi operazione di riassegnazione dentro la funzione fa puntare la variabile locale a un NUOVO oggetto, lasciando invariato l'originale esterno.
- Se l'oggetto è **mutabile** (`list` , `dict` , `set`), modificarne lo stato interno (es. con `.append()` o assegnando un indice) **altera l'oggetto originale anche fuori dalla funzione**!

```
flowchart TD
    subgraph CHIAMANTE ["Ambiente Esterno"]
        NUM["x = 10 (int: immutabile)"]
        LST["lista = [1, 2] (list: mutabile)"]
    end

    subgraph FUNZIONE ["All'interno della Funzione"]
        F_NUM["x += 1\nCrea NUOVO intero 11\nL'originale 10 RESTA INVARIATO"]
        F_LST["lista.append(3)\nModifica lo STESSO oggetto nello Heap!\nL'originale
    end

    NUM -.-> F_NUM
    LST --> F_LST

    style F_NUM fill:#d4edda,stroke:#28a745,color:#155724
    style F_LST fill:#f8d7da,stroke:#dc3545,color:#721c24
```

---

5. Scope e Visibilità: La Regola LEGB

Quando Python incontra una variabile dentro una funzione, la cerca secondo la gerarchia rigorosa **LEGB**:

```
flowchart LR
    L["1. Local (L)\nInterna alla funzione"] --> E["2. Enclosing (E)\nFunzione conte
    E --> G["3. Global (G)\nModulo a livello di file"]
    G --> B["4. Built-in (B)\nNomi predefiniti (len, print, range...)"]

    style L fill:#cce5ff,stroke:#007bff,color:#004085
    style E fill:#d4edda,stroke:#28a745,color:#155724
    style G fill:#fff3cd,stroke:#ffc107,color:#856404
    style B fill:#f8d7da,stroke:#dc3545,color:#721c24
```

La Parola Chiave `global`

In lettura possiamo leggere variabili globali senza problemi. Ma se tentiamo di **assegnare** un valore a una variabile globale dentro una funzione, Python creerà una nuova variabile locale, a meno di non usare la keyword `global`:

```
contatore = 0

def incrementa():
    global contatore  # Dichiara esplicitamente che vogliamo modificare la
    contatore += 1

incrementa()
print(contatore)  # 1
```

> ⚠️ **Best Practice**
>
> L'uso di `global` è fortemente sconsigliato nei progetti seri perché crea effetti collaterali e rende il codice difficile da debuggare. Preferisci sempre passare i dati come argomenti e restituire il nuovo valore con `return`.

---

6. Funzioni Lambda (Funzioni Anonime)

Una **lambda** è una funzione compatta e monoriga, definita senza un nome tramite la sintassi: lambda parametri : espressione

```
# Lambda che raddoppia un numero
doppio = lambda x: x * 2
print(doppio(5))  # 10

# Uso tipico: criterio di ordinamento personalizzato per una lista
studenti = [("Alice", 28), ("Bob", 22), ("Carla", 30)]
# Ordina per voto (secondo elemento della tupla)
studenti.sort(key=lambda s: s[1], reverse=True)
print(studenti)  # [('Carla', 30), ('Alice', 28), ('Bob', 22)]
```

7. Domande d’Esame e Concetti Chiave

❓ **Domande di Verifica**

1. *Cosa succede se dichiaro un parametro di default come lista vuota:* `def func(v=[]):` *?*
   Risposta: La lista viene creata solo all’interpretazione iniziale della funzione. Tutte le successive invocazioni che omettono l’argomento condivideranno la medesima lista in memoria, accumulando dati in modo anomalo.
2. *Qual è la differenza pratica tra* `*args` *e* `**kwargs` *?*
   Risposta: `*args` impacchetta un numero imprecisato di argomenti posizionali in una tupla ordinata; `**kwargs` impacchetta argomenti con nome (chiave=valore) in un dizionario.
3. *Cosa stabilisce la regola LEGB per la risoluzione dei nomi?*
   Risposta: Stabilisce la sequenza di ricerca: cerca prima nello scope Locale, poi nelle funzioni Racchiudenti (Enclosing), poi nello scope Globale del modulo e infine nei Built-in del linguaggio.

---

8. Programma Esecutivo Completo

🍼 **Programma Completo: Motore di Calcolo Matematico Modulare**

```python
def statistiche(*numeri: float, arrotonda: int = 2, **metadati) -> di
    """
    Calcola statistiche su un numero arbitrario di cifre e allega met
    Dimostra l'uso congiunto di *args, default kwargs e **kwargs.
    """
    if not numeri:
        return {"errore": "Nessun numero fornito"}

    totale: float = sum(numeri)
    media: float = totale / len(numeri)
    val_min: float = min(numeri)
    val_max: float = max(numeri)

    report = {
        "conteggio": len(numeri),
        "somma": round(totale, arrotonda),
        "media": round(media, arrotonda),
        "min": round(val_min, arrotonda),
        "max": round(val_max, arrotonda),
        "info_aggiuntive": metadati
    }
    return report

if __name__ == "__main__":
    risultato = statistiche(
        14.555, 22.3, 8.91, 100.25, 4.0,
        arrotonda=1,
        operatore="Prof. Caseti",
        materia="Informatica",
        sessione="Esami 2026"
    )

    print("=== RISULTATO ELABORAZIONE DATI ===")
    for chiave, valore in risultato.items():
        print(f"{chiave:<16}: {valore}")
```

> **In Sintesi (A Colpo d'Occhio)**
>
> - **Niente Puntatori Espliciti, ma Riferimenti Ovunque**: in Python ogni variabile è un **riferimento opaco ad un `PyObject` nello Heap**.
> - **Indirizzo di Memoria**: la funzione **`id(x)`** restituisce l'indirizzo fisico univoco dell'oggetto (visualizzabile in esadecimale con `hex(id(x))`).
> - **`==` vs `is`**:
>   - `==` verifica l'**uguaglianza di valore** (hanno lo stesso contenuto?).
>   - `is` verifica l'**identità di memoria** (`id(a) == id(b)`: sono lo stesso identico oggetto fisico?).
> - **Tipi Mutabili vs Immutabili**:
>   - **Immutabili** (`int`, `float`, `str`, `tuple`): modificarli genera un nuovo oggetto.
>   - **Mutabili** (`list`, `dict`, `set`): modifiche sul posto (*in-place*).
> - **Copia di Oggetti**: `b = a` copia solo il puntatore; usa **`copy.copy()`** per la shallow copy e **`copy.deepcopy()`** per duplicare strutture annidate.
> - **Garbage Collector**: basato principalmente su **Reference Counting** (deallocazione immediata a contatore zero) assistito da un ciclo-rilevatore per i riferimenti circolari.

---

1. Ci sono i Puntatori in Python?

Gli studenti provenienti dal C++ spesso chiedono: *"Dov'è finito l'operatore asterisco `*` e l'operatore ampersand `&`?"*.

La risposta è: **in Python non puoi manipolare direttamente i puntatori, ma ogni singola variabile è già, concettualmente, un puntatore.**

Sotto il cofano in CPython, qualsiasi dato è una struttura C chiamata `PyObject`:

```
// Struttura concettuale interna di CPython
struct _object {
    _PyObject_HEAD_EXTRA // Riferimenti per la lista del GC
    Py_ssize_t ob_refcnt; // CONTATORE DI RIFERIMENTI (Reference Count)
    struct _typeobject *ob_type; // PUNTATORE AL TIPO DELL'OGGETTO
};
```

Quando creiamo `x = 1000`, la variabile `x` non contiene il numero 1000: contiene l'**indirizzo di memoria** della struct `PyObject` allocata nello Heap.

Ispezionare la RAM: La Funzione `id()`

La funzione nativa `id(oggetto)` restituisce l'identificatore univoco dell'oggetto, che in CPython corrisponde esattamente al suo **indirizzo fisico di memoria RAM**:

```
x = 1000
print(f"Indirizzo decimale: {id(x)}")
print(f"Indirizzo esadecimale RAM: {hex(id(x))}") # es. 0x7fa8102b3f50
```

---

2. La Differenza Vitale: `==` vs `is`

Uno degli errori logici più subdoli nei programmi Python è confondere l'uguaglianza con l'identità:

```
flowchart TD
    subgraph IDENTITA ["Identità di Memoria (is)"]
        V1["a = [1, 2]"] --> OBJ1["Oggetto Lista A\nIndirizzo: 0x100\nContenuto: [1,
        V2["b = [1, 2]"] --> OBJ2["Oggetto Lista B\nIndirizzo: 0x200\nContenuto: [1,
    end

    style OBJ1 fill:#d4edda,stroke:#28a745,color:#155724
    style OBJ2 fill:#fff3cd,stroke:#ffc107,color:#856404
```

Operatore	Cosa Confronta?	Metodo Interno Invocato	Domanda Chiave
`==`	**Valore (Contenuto)**	`a.__eq__(b)`	*I due oggetti hanno lo stesso contenuto logico?*
`is`	**Identità (Indirizzo RAM)**	`id(a) == id(b)`	*Le due etichette puntano allo stesso identico oggetto fisico?*

Esempio Pratico:

```
lista1 = [1, 2, 3]
lista2 = [1, 2, 3]
lista3 = lista1

print(lista1 == lista2)  # True  (hanno gli stessi numeri all'interno)
print(lista1 is lista2)  # False (sono due liste DISTINTE in memoria RAM!)

print(lista1 is lista3)  # True  (puntano allo STESSO identico blocco di me
```

> 🎯 **Regola d'Oro per il confronto con `None`**
>
> Per verificare se una variabile è vuota o nulla, usa **sempre** `is None` (o `is not None`) e mai `== None`. `None` è un singleton garantito in tutta la vita del processo Python!

---

3. Mutabilità vs Immutabilità

Comprendere la mutabilità è fondamentale per non introdurre bug invisibili:

```mermaid
flowchart LR
    subgraph IMMUTABILE ["Immutabile (es. int, str)"]
        direction TB
        X["x = 5"] --> O1["Oggetto 5 (0x10)"]
        X2["x = x + 1"] --> O2["NUOVO Oggetto 6 (0x20)"]
    end

    subgraph MUTABILE ["Mutabile (es. list, dict)"]
        direction TB
        L["lst = [1]"] --> M1["Oggetto Lista (0x99)\n[1]"]
        L2["lst.append(2)"] --> M1
    end

    style O1 fill:#e1f5fe,stroke:#03a9f4,color:#01579b
    style O2 fill:#ffe873,stroke:#ffd43b,color:#000
    style M1 fill:#d4edda,stroke:#28a745,color:#155724
```

- **Tipi Immutabili** (`int` , `float` , `str` , `tuple` , `bool`): non possono cambiare stato. Qualsiasi alterazione crea un nuovo oggetto in un nuovo indirizzo RAM.
- **Tipi Mutabili** (`list` , `dict` , `set` , istanze di classi): possono essere modificati direttamente nella loro cella di memoria originale.

---

4. Copia di Oggetti: Assegnazione, Copia Superficiale e Profonda

Quando devi duplicare una collezione mutabile, l'operatore di assegnazione `=` **NON copia i dati**, ma copia solo il riferimento!

```
# 1. Assegnazione semplice (effetto specchio):
a = [1, 2, 3]
b = a
b.append(99)
print(a)  # [1, 2, 3, 99] -> Anche 'a' è stata modificata!
```

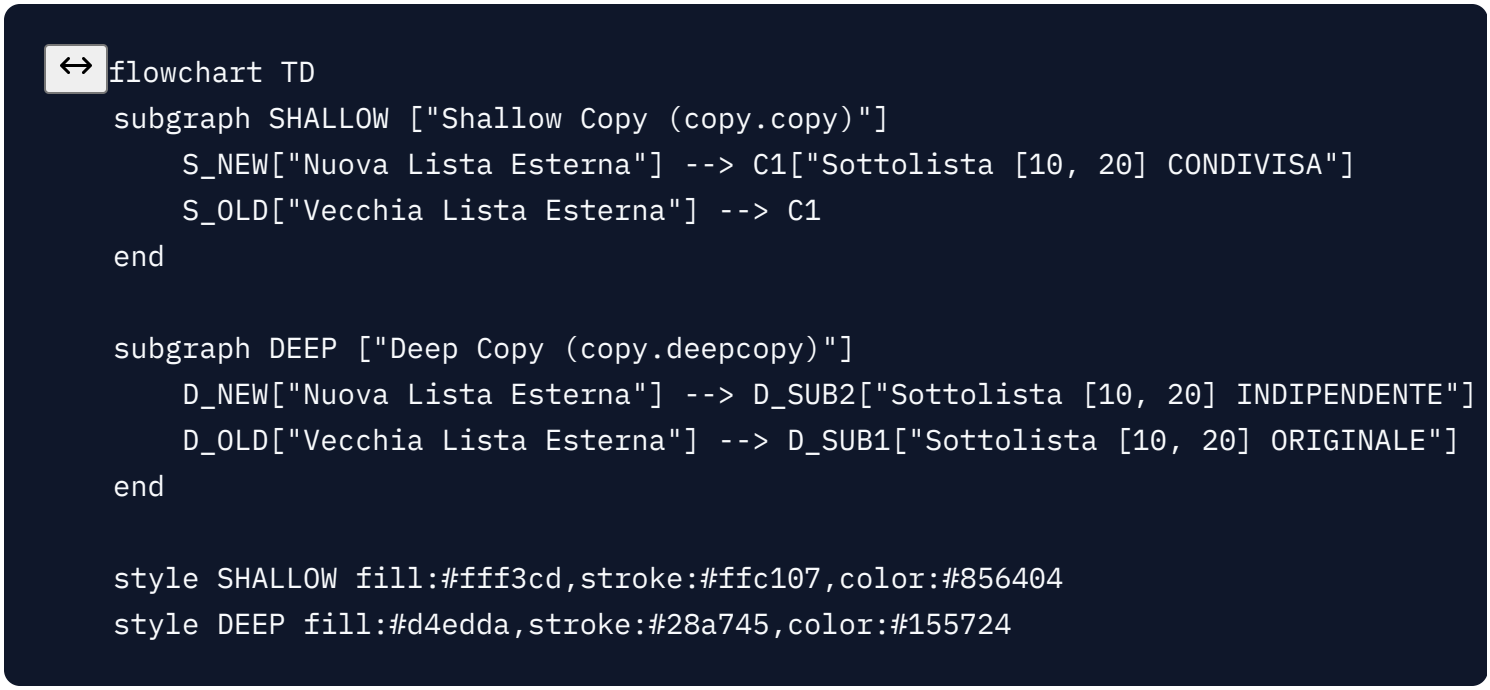
Per ottenere copie reali indipendenti usiamo il modulo nativo `copy` :

```
import copy

originale = [[10, 20], [30, 40]]

# 2. Shallow Copy (Copia Superficiale):
superficiale = copy.copy(originale)  # oppure: originale[:]
# Copia la lista esterna, MA le sottoliste interne sono ancora condivise!
superficiale[0][0] = 999
print(originale[0][0])  # 999 -> Il dato interno è cambiato anche nell'orig

# 3. Deep Copy (Copia Profonda Totale):
profonda = copy.deepcopy(originale)
profonda[0][0] = 111
print(originale[0][0])  # 999 -> L'originale è TOTALMENTE PROTETTO!
```



---

5. Come Funziona il Garbage Collector in Python

Python gestisce la memoria automaticamente tramite due meccanismi complementari:

A. Reference Counting (Conteggio dei Riferimenti)

- È il meccanismo primario.

- Ogni oggetto memorizza quante variabili o collezioni lo stanno referenziando.
- **Nel momento esatto in cui il contatore scende a 0**, la memoria occupata dall'oggetto viene liberata e restituita all'allocatore di sistema immediatamente.

```
import sys

x = ["oggetto", "di", "test"]
print(sys.getrefcount(x) - 1)  # 1 riferimento (la variabile x)

y = x
print(sys.getrefcount(x) - 1)  # 2 riferimenti (x e y)

del y  # Rimuove il riferimento y
print(sys.getrefcount(x) - 1)  # Torna a 1
```

B. Cyclic Garbage Collector (Rilevatore di Cicli)

Cosa succede se due oggetti si puntano a vicenda ma non sono più raggiungibili dal programma?

```
a = []
b = []
a.append(b)  # 'a' punta a 'b'
b.append(a)  # 'b' punta a 'a'
del a
del b
# I loro reference count non arrivano mai a 0 a causa del ciclo!
```

Per evitare *memory leaks*, interviene il **Garbage Collector generazionale** (modulo `gc`), che periodicamente ispeziona lo Heap, individua isole isolate di riferimenti circolari e le distrugge forzatamente.

---

6. Domande d'Esame e Concetti Chiave

> ❓ **Domande di Verifica**

1. *Qual è la differenza fondamentale tra l'operatore `==` e l'operatore `is`?*
   Risposta: `==` verifica se i due operandi hanno lo stesso valore o contenuto logico chiamando `__eq__()`. L'operatore `is` verifica se puntano allo stesso indirizzo di memoria fisico in RAM (`id(a) == id(b)`).
2. *Quando è strettamente necessario usare `copy.deepcopy()` al posto di una copia normale?*
   Risposta: Quando la struttura da duplicare contiene elementi mutabili annidati (come liste di liste o dizionari di dizionari); una shallow copy duplicherebbe solo il contenitore esterno lasciando condivisi gli oggetti interni.
3. *Come gestisce Python la memoria quando una variabile viene cancellata con `del`?*
   Risposta: L'istruzione `del` decrementa di 1 il reference counter dell'oggetto. Se il contatore raggiunge lo zero, l'oggetto viene deallocato istantaneamente dal motore CPython.

---

7. Programma Esecutivo Completo

💉 Programma Completo: Ispezione Pratica di Memoria e Riferimenti

```
import copy
import sys

def dimostra_memoria():
    print("=== 1. ANALISI IDENTITÀ E INDIRIZZI RAM ===")
    lista_a = [10, 20, 30]
    lista_b = [10, 20, 30]
    lista_c = lista_a

    print(f"lista_a -> Indirizzo: {hex(id(lista_a))} | Valore: {lista
    print(f"lista_b -> Indirizzo: {hex(id(lista_b))} | Valore: {lista
    print(f"lista_c -> Indirizzo: {hex(id(lista_c))} | Valore: {lista

    print(f"\nlista_a == lista_b: {lista_a == lista_b} (Stesso valore
    print(f"lista_a is lista_b: {lista_a is lista_b} (Oggetti fisici
    print(f"lista_a is lista_c: {lista_a is lista_c} (Stesso identico

    print("\n=== 2. DIMOSTRAZIONE DI SHALLOW VS DEEP COPY ===")
    registro_originale = [["Mario", 8], ["Luigi", 7]]

    copia_superficiale = copy.copy(registro_originale)
    copia_profonda = copy.deepcopy(registro_originale)

    # Modifichiamo il voto di Mario nella copia superficiale
    copia_superficiale[0][1] = 10

    print(f"Originale dopo mod shallow: {registro_originale} (MODIFIC

    # Modifichiamo il voto di Luigi nella copia profonda
    copia_profonda[1][1] = 4
    print(f"Originale dopo mod deep:    {registro_originale} (PROTETT
    print(f"Copia profonda indipendente: {copia_profonda}")

if __name__ == "__main__":
    dimostra_memoria()
```

> **In Sintesi (A Colpo d'Occhio)**
>
> - **Classe e Istanza**: lo stampo concettuale (`class`) e l'oggetto concreto in memoria.
> - **Costruttore** `__init__` : metodo speciale eseguito automaticamente alla creazione dell'oggetto.
> - **Il parametro** `self` : riferimento esplicito all'istanza corrente (l'equivalente del puntatore `this->` in C++).
> - **Incapsulamento**: nessuna parola chiave rigida (`public` / `private`); convenzione con `_nome` (protetto) e `__nome` (privato con *Name Mangling*).
> - `@property` : crea getter e setter eleganti mantenendo la sintassi ad accesso diretto (`obj.saldo = 100`).
> - **Dunder Methods**: metodi speciali con doppio underscore (`__str__` , `__len__` , `__eq__`) per integrare gli oggetti con la sintassi nativa di Python.
> - `@dataclass` : crea classi dati complete e auto-documentate in 3 righe (la moderna alternativa alle `struct` di C++).

---

1. Classi, Istanze e il Ruolo di `self`

In Python definiamo una classe con la parola chiave `class` . Per convenzione PEP 8, i nomi delle classi usano la notazione **PascalCase** (es. `ContoBancario`).

```
classDiagram
    class Veicolo {
        +marca: str
        +modello: str
        +anno: int
        +mostra_scheda()
    }
    class Auto {
        +porte: int
        +mostra_scheda()
    }
    Veicolo <|-- Auto : Eredita
```

Il Costruttore `__init__` e il Parametro `self`

```
class Automobile:
    # Costruttore (inizializzatore dello stato dell'oggetto)
    def __init__(self, marca: str, modello: str, cavalli: int):
        self.marca = marca       # Attributo d'istanza
        self.modello = modello   # Attributo d'istanza
        self.cavalli = cavalli   # Attributo d'istanza

    # Metodo di istanza
    def scheda_tecnica(self) -> str:
        return f"{self.marca} {self.modello} ({self.cavalli} CV)"

# Istanziazione (NON si usa la keyword 'new' come in C++ o Java!)
mia_auto = Automobile("Alfa Romeo", "Giulia", 280)
print(mia_auto.scheda_tecnica())
```

> ❓ **Perché `self` deve essere sempre il primo parametro?**
>
> In C++, il puntatore `this` viene passato implicitamente dietro le quinte. In Python, coerentemente con il principio dello Zen *"Explicit is better than implicit"*, il riferimento all'oggetto viene dichiarato esplicitamente nella firma del metodo come `self`. Quando chiami `mia_auto.scheda_tecnica()`, Python la traduce automaticamente in `Automobile.scheda_tecnica(mia_auto)`.

---

2. Attributi di Istanza vs Attributi di Classe

- **Attributi di Istanza**: definiti dentro `__init__` con `self.nome`; ogni singolo oggetto ha i propri valori indipendenti.
- **Attributi di Classe**: definiti a livello di corpo classe (fuori da qualsiasi metodo); sono variabili condivise da **tutte le istanze** (l'equivalente delle variabili `static` del C++).

```
class Utente:
    contatore_iscritti = 0  # Attributo di classe condiviso!

    def __init__(self, username: str):
        self.username = username  # Attributo di singola istanza
        Utente.contatore_iscritti += 1

u1 = Utente("Alessandro")
u2 = Utente("Beatrice")
print(Utente.contatore_iscritti)  # 2 (condiviso da entrambi!)
```

---

3. Incapsulamento Pythonico e Name Mangling

La filosofia di Python è pragmatica: *"We are all consenting adults here"* (siamo tutti adulti consenzienti). Non esistono blocchi invalicabili imposti dal compilatore, ma convenzioni sintattiche ben definite:

1. **Pubblico (`nome`)**: accessibile liberamente da chiunque.
2. **Protetto (`_nome`)**: convenzione che indica agli sviluppatori che la variabile è destinata a uso interno; non andrebbe modificata dall'esterno.
3. **Privato (`__nome`)**: attiva il **Name Mangling**. L'interprete rinomina internamente la variabile in `_NomeClasse__nome` per evitare collisioni accidentali nelle sottoclassi:

```
class Cassaforte:
    def __init__(self, combinazione: str):
        self.__combinazione = combinazione  # Privato con name mangling

c = Cassaforte("1234")
# print(c.__combinazione)  <-- AttributeError: 'Cassaforte' object has no a

# In realtà Python l'ha solo rinominata:
print(c._Cassaforte__combinazione)  # "1234" (raggiungibile se si sa dove c
```

---

4. Proprietà Avanzate: `@property` (Getter e Setter)

In linguaggi rigidi si è costretti a scrivere metodi verbosi come `get_stipendio()` e `set_stipendio()`.
In Python, il decoratore **`@property`** permette di incapsulare logica e controlli di validazione mantenendo all'esterno la sintassi naturale di un semplice attributo:

```
class Termometro:
    def __init__(self, gradi: float = 0.0):
        self._celsius = gradi

    @property
    def celsius(self) -> float:
        """Getter: invocato quando si legge t.celsius"""
        return self._celsius

    @celsius.setter
    def celsius(self, valore: float):
        """Setter: invocato quando si assegna t.celsius = ..."""
        if valore < -273.15:
            raise ValueError("Temperatura impossibile: inferiore allo Zero
        self._celsius = valore

t = Termometro(25.0)
print(t.celsius)     # Chiama il getter -> 25.0
t.celsius = 30.5     # Chiama il setter -> aggiorna con successo
# t.celsius = -300   # Blocca l'esecuzione e solleva ValueError!
```

---

5. Metodi Speciali (Dunder Methods)

I metodi circondati da doppio underscore (*Double Underscore*, detti colloquialmente **dunder**) consentono di insegnare a Python come interagire con le tue classi personalizzate:

Dunder Method	Quando Viene Invocato?	Equivalente C++
`__str__(self)`	Invocato da `print(obj)` o `str(obj)` per gli utenti	`friend ostream& operator<<`
`__repr__(self)`	Rappresentazione tecnica per programmatori e log di debug	Debugger inspector
`__len__(self)`	Invocato dalla funzione `len(obj)`	Metodo `.size()`
`__eq__(self, altro)`	Invocato dall'operatore di uguaglianza `obj1 == obj2`	`operator==`
`__lt__(self, altro)`	Invocato dall'operatore minore `obj1 < obj2` (usato da `sort()`)	`operator<`

```
class Libro:
    def __init__(self, titolo: str, pagine: int):
        self.titolo = titolo
        self.pagine = pagine

    def __str__(self):
        return f"Libro: '{self.titolo}' ({self.pagine} pag.)"

    def __len__(self):
        return self.pagine

libro = Libro("Il Nome della Rosa", 512)
print(libro)       # Chiama __str__ -> Libro: 'Il Nome della Rosa' (512 pag
print(len(libro))  # Chiama __len__ -> 512
```

---

6. Ereditarietà e Chiamata a `super()`

L'ereditarietà si esprime passando la classe base tra le parentesi tonde della sottoclasse:

```
class Persona:
    def __init__(self, nome: str, codice_fiscale: str):
        self.nome = nome
        self.codice_fiscale = codice_fiscale

    def descrivi(self):
        return f"Persona: {self.nome} ({self.codice_fiscale})"

class Studente(Persona):
    def __init__(self, nome: str, codice_fiscale: str, matricola: int):
        # Invocazione del costruttore genitore tramite super()
        super().__init__(nome, codice_fiscale)
        self.matricola = matricola

    # Overriding (sovrascrittura) del metodo descrivi
    def descrivi(self):
        return f"Studente: {self.nome}, Matr: {self.matricola}"
```

---

7. Strutture Dati Moderne: Le `@dataclass`

In C++ per raggruppare dati si usa spesso una `struct`. In Python moderno (da Python 3.7+), la libreria standard offre il modulo **`dataclasses`**: con il decoratore `@dataclass`, Python scrive automaticamente al posto tuo il costruttore `__init__`, la stampa formattata `__repr__` e il confronto `__eq__`!

```
from dataclasses import dataclass

@dataclass
class Prodotto:
    nome: str
    prezzo: float
    scorte: int = 0

# Genera automaticamente __init__, __repr__ e __eq__!
p1 = Prodotto("Tastiera Meccanica", 89.99, 12)
p2 = Prodotto("Tastiera Meccanica", 89.99, 12)

print(p1)          # Prodotto(nome='Tastiera Meccanica', prezzo=89.99, sco
print(p1 == p2)    # True (confronto automatico di tutti i campi!)
```

> 🎯 **Quando Usare le Dataclass?**
>
> Usa sempre `@dataclass` quando crei classi che fungono primariamente da contenitori di dati strutturati (es. record di database, pacchetti di rete, configurazioni, coordinate geometriche).

---

8. Domande d'Esame e Concetti Chiave

> ❓ **Domande di Verifica**
>
> 1. *Qual è la differenza fondamentale tra attributi di istanza e attributi di classe?*
>   **Risposta**: Gli attributi di istanza (`self.x`) appartengono al singolo oggetto e ciascuna istanza possiede una copia indipendente. Gli attributi di classe sono dichiarati nel corpo della classe e sono condivisi tra tutte le istanze dello stesso tipo.
> 2. *A cosa serve la funzione `super()` nell'ereditarietà?*
>   **Risposta**: Restituisce un proxy temporaneo che permette di delegare chiamate ai

metodi della classe genitore, garantendo la corretta inizializzazione dello stato ereditato.

3. *Cosa realizza tecnicamente il decoratore* `@property` *?*
   Risposta: Trasforma un metodo in un getter di sola lettura (o con validazione se abbinato a un `@setter`), consentendo di accedervi come se fosse un normale attributo senza parentesi tonde.

---

9. Programma Esecutivo Completo

🧪 Programma Completo: Sistema di Gestione Conto Bancario con Eccezioni e Dunder

```
class SaldoInsufficienteError(Exception):
    """Eccezione personalizzata per prelievi oltre il limite."""
    pass

class ContoBancario:
    tasso_interesse_base: float = 0.015  # Attributo di classe (1.5%)

    def __init__(self, intestatario: str, iban: str, saldo_iniziale: 
        self.intestatario = intestatario
        self.iban = iban
        self._saldo = max(0.0, saldo_iniziale)
        self._movimenti: list[str] = [f"Apertura conto con saldo €{sel

    @property
    def saldo(self) -> float:
        """Getter sicuro per la consultazione del saldo."""
        return self._saldo

    def deposita(self, importo: float) -> None:
        if importo <= 0:
            raise ValueError("L'importo del deposito deve essere posit
        self._saldo += importo
        self._movimenti.append(f"Deposito: +€{importo:.2f}")

    def preleva(self, importo: float) -> None:
        if importo <= 0:
            raise ValueError("L'importo da prelevare deve essere posit
        if importo > self._saldo:
            raise SaldoInsufficienteError(f"Impossibile prelevare €{im
        self._saldo -= importo
        self._movimenti.append(f"Prelievo: -€{importo:.2f}")

    def __str__(self) -> str:
        return f"Conto [{self.iban}] di {self.intestatario} | Saldo: €

    def __len__(self) -> int:
        return len(self._movimenti)

if __name__ == "__main__":
    conto = ContoBancario("Alessandro Caseti", "IT99X0123456789012345
    conto.deposita(250.50)
```

```
conto.preleva(100.0)

print(conto)
print(f"Numero totale di operazioni registrate: {len(conto)}")

try:
    conto.preleva(5000.0)  # Tentativo di prelievo oltre limite
except SaldoInsufficienteError as errore:
    print(f"⚠️ Operazione negata: {errore}")
```

> **In Sintesi (A Colpo d'Occhio)**
>
> - **Regola d'Oro**: usa **SEMPRE** il Context Manager `with open(...) as f:` (garantisce la chiusura immediata del file anche in caso di crash).
> - **Encoding Esplicito**: specifica sempre `encoding="utf-8"` per evitare bug di caratteri accentati (soprattutto su sistemi Windows).
> - **Modalità Principali**: `'r'` (lettura), `'w'` (scrittura distruttiva da zero), `'a'` (append in coda), `'b'` (binario per immagini/PDF).
> - **Lettura ad Alta Efficienza**: per file di grandi dimensioni usa `for riga in f:` (legge riga per riga senza saturare la RAM).
> - **Gestione Eccezioni**: blocco `try - except FileNotFoundError - finally`.
> - **Formati Strutturati Standard**: moduli integrati `json` (`json.dump` / `json.load`) e `csv` (`csv.DictReader`).

---

1. Il Context Manager: `with open(...)`

In C++ per gestire i file si istanziano oggetti `ifstream` o `ofstream` e si invoca `.close()`. Se si verifica un'eccezione prima del `close()`, il file rischia di rimanere bloccato dal sistema operativo o corrotto.

In Python, la sintassi idiomatica impone l'uso del **Context Manager** tramite la parola chiave `with`:

```
flowchart TD
    A["with open('dati.txt', 'r') as f:"] --> B["1. Apertura File Descriptor\n(invoc
    B --> C["2. Esecuzione Istruzioni nel Blocco\n(lettura / elaborazione / calcoli)
    C -->|Uscita normale o ECCEZIONE| D["3. Chiusura Automatica Garantita\n(invoca i

    style A fill:#2b5b84,stroke:#306998,color:#fff
    style B fill:#cce5ff,stroke:#007bff,color:#004085
    style C fill:#d4edda,stroke:#28a745,color:#155724
    style D fill:#d1ecf1,stroke:#17a2b8,color:#0c5460
```

> 🎯 **Regola d'Oro**
>
> Non chiamare mai `open()` in modo "nudo" (`f = open(...)`). L'istruzione `with open(...) as f:` garantisce che il file venga **sempre chiuso**, anche in caso di errori imprevisti a runtime.

---

2. Modalità di Apertura ed Encoding

La funzione `open()` accetta il percorso del file, la modalità e l'encoding:
open(percorso, mode =′ r′, encoding =′ utf − 8′)

Modalità	Nome	Comportamento se il file ESISTE	Comportamento se NON esiste
`'r'`	Read (Default)	Apre il cursore all'inizio per la lettura	**Solleva** `FileNotFoundError`
`'w'`	Write	**Sovrascrive e azzera il contenuto!**	Crea un nuovo file vuoto
`'a'`	Append	Sposta il cursore alla fine (aggiunge in coda)	Crea un nuovo file vuoto
`'r+'`	Read + Write	Consente sia lettura che scrittura in-place	**Solleva** `FileNotFoundError`
`'b'`	Binary	Modalità binaria (es. `'rb'`, `'wb'` per immagini, zip)	Dipende dalla lettera associata

> ⚠️ **La Trappola dell'Encoding su Windows**

> Se ometti `encoding="utf-8"`, Windows utilizzerà la codifica legacy di sistema (spesso `cp1252`), causando crash con errori del tipo `UnicodeDecodeError` quando il file contiene lettere accentate (`è`, `à`, `ù`) o emoji!

---

3. Tecniche di Lettura del Testo

Python offre diversi modi per consumare il contenuto di un file di testo:

1. Iterazione Diretta Riga per Riga (Metodo Consigliato)

È il metodo più efficiente in assoluto perché legge le righe una alla volta come un flusso (*stream*), consumando pochissima memoria anche con file da diversi gigabyte:

```
with open("studenti.txt", "r", encoding="utf-8") as f:
    for riga in f:
        # riga.strip() rimuove spazi bianchi e il terminatore \n
        print(f"Letta: {riga.strip()}")
```

2. Lettura Completa con `.read()` o `.readlines()`

```
with open("config.txt", "r", encoding="utf-8") as f:
    # f.read() carica l'INTERO contenuto in un'unica stringa
    tutto_il_testo = f.read()

with open("lista.txt", "r", encoding="utf-8") as f:
    # f.readlines() carica tutte le righe in una lista di stringhe
    righe = f.readlines()
```

---

4. Tecniche di Scrittura

Per scrivere usiamo `.write()` o `.writelines()`:

```
nomi = ["Mario Rossi", "Giulia Bianchi", "Luca Verdi"]

with open("elenco.txt", "w", encoding="utf-8") as f:
    for nome in nomi:
        # Nota: write() NON aggiunge il carattere a-capo in automatico!
        f.write(f"{nome}\n")
```

---

5. Gestione Robusta degli Errori: `try - except - finally`

La gestione dei file è intrinsecamente soggetta a errori di sistema (file mancante, permessi negati, disco pieno). Un programma professionale deve gestire queste situazioni con eleganza:

```
try:
    with open("dati_segreti.txt", "r", encoding="utf-8") as f:
        contenuto = f.read()
except FileNotFoundError:
    print("⚠️ Errore: Il file richiesto non esiste sul percorso specificat
except PermissionError:
    print("🚫 Errore: Permessi insufficienti per leggere il file!")
except Exception as e:
    print(f"❌ Errore imprevisto durante l'I/O: {e}")
else:
    print("✅ File letto con successo!")
finally:
    print("Pulizia e termine della procedura.")
```

---

6. Formati Strutturati: JSON e CSV

Nei progetti reali i dati non sono quasi mai solo semplice testo grezzo, ma record formattati. La libreria standard di Python include moduli integrati per gestirli senza installare librerie esterne.

A. Il Modulo `json` (Dati Strutturati e API Web)

```
import json

dati_utente = {
    "username": "casetti_a",
    "punteggio": 1500,
    "preferenze": {"tema": "dark", "notifiche": True}
}

# 1. Scrittura su file JSON: json.dump()
with open("utente.json", "w", encoding="utf-8") as f:
    json.dump(dati_utente, f, indent=4)

# 2. Lettura da file JSON: json.load()
with open("utente.json", "r", encoding="utf-8") as f:
    dati_letti = json.load(f)
    print(dati_letti["preferenze"]["tema"])  # dark
```

B. Il Modulo `csv` (Tabelle e Fogli di Calcolo)

```
import csv

# Scrittura di un file CSV con DictWriter
studenti = [
    {"Matricola": "A101", "Nome": "Sara", "Voto": 28},
    {"Matricola": "A102", "Nome": "Davide", "Voto": 30}
]

with open("registro.csv", "w", newline="", encoding="utf-8") as f:
    campi = ["Matricola", "Nome", "Voto"]
    writer = csv.DictWriter(f, fieldnames=campi)
    writer.writeheader()
    writer.writerows(studenti)

# Lettura con DictReader
with open("registro.csv", "r", encoding="utf-8") as f:
    reader = csv.DictReader(f)
    for riga in reader:
        print(f"Studente: {riga['Nome']} (Voto: {riga['Voto']})")
```

---

7. Domande d’Esame e Concetti Chiave

❓ Domande di Verifica

1. *Perché è fondamentale usare `with open(...)` rispetto a `open()` semplice?*
   Risposta: Perché il costrutto `with` sfrutta il protocollo del Context Manager, assicurando che la chiamata di chiusura `.close()` sul file avvenga in modo garantito al termine del blocco, perfino se si verifica un crash o viene sollevata un’eccezione.
2. *Qual è il vantaggio dell’iterazione diretta `for riga in f:` rispetto a `f.readlines()`?*
   Risposta: `f.readlines()` carica tutte le righe del file simultaneamente nella RAM sotto forma di lista, rischiando di saturare la memoria in caso di file molto grandi. L’iterazione `for riga in f:` carica una sola riga alla volta sfruttando un generatore a flusso.
3. *Cosa fa la modalità `'w'` se il file esiste già?*
   Risposta: Troncamento immediato: il contenuto preesistente del file viene completamente cancellato e azzerato senza richiedere conferme.

---

8. Programma Esecutivo Completo

🧪 Programma Completo: Gestore di Log e Salvataggio JSON

```
import json
import os
from datetime import datetime

FILE_CONFIG = "sessione.json"
FILE_LOG = "eventi.log"

def registra_evento(messaggio: str):
    timestamp = datetime.now().strftime("%Y-%m-%d %H:%M:%S")
    with open(FILE_LOG, "a", encoding="utf-8") as f:
        f.write(f"[{timestamp}] {messaggio}\n")

def salva_stato(dati: dict):
    with open(FILE_CONFIG, "w", encoding="utf-8") as f:
        json.dump(dati, f, indent=4, ensure_ascii=False)
    registra_evento("Salvataggio stato configurazione completato.")

def carica_stato() -> dict:
    if not os.path.exists(FILE_CONFIG):
        registra_evento("File configurazione non trovato. Creato stat
        return {"accessi": 0, "ultimo_utente": "Ospite"}

    try:
        with open(FILE_CONFIG, "r", encoding="utf-8") as f:
            stato = json.load(f)
            registra_evento("Stato configurazione caricato con succes
            return stato
    except Exception as e:
        registra_evento(f"Errore lettura JSON: {e}")
        return {"accessi": 0, "ultimo_utente": "Ripristino Fallback"}

if __name__ == "__main__":
    stato = carica_stato()
    stato["accessi"] += 1
    stato["ultimo_utente"] = "Alessandro Caseti"

    salva_stato(stato)
    print(f"Operazione completata. Accessi totali: {stato['accessi']}
```

> **In Sintesi (A Colpo d'Occhio)**
>
> - **Ricerca**:
> - **Lineare**: O(n), funziona su qualsiasi lista; implementata nativamente con **`elem in lista`**.
> - **Binaria (Dicotomica)**: O(log n), richiede lista **pre-ordinata**; disponibile nativamente con **`import bisect`**.
> - **Didattica dell'Ordinamento**: Bubble Sort e Selection Sort implementabili in poche righe grazie allo scambio pythonico **`a, b = b, a`**.
> - **Timsort (Lo Standard di Sistema)**: algoritmo ibrido (Merge Sort + Insertion Sort) stabile con complessità da **$O(n)$** nel caso migliore a **$O(n \log n)$** nel caso peggiore.
> - **`sort()` vs `sorted()`**:
> - **`lista.sort()`**: ordina **sul posto** (*in-place*), muta l'originale e restituisce `None`.
> - **`sorted(iterabile)`**: crea e restituisce una **nuova lista ordinata**, lasciando l'originale intatto.
> - **Ordinamento Avanzato**: uso del parametro **`key=lambda x: ...`** per ordinare collezioni complesse per campi specifici.

---

1. Algoritmi di Ricerca: Lineare vs Binaria

```
flowchart TD
    A["Algoritmo di Ricerca"]
    A --> L["Ricerca Lineare\nComplessità: O(n)\nNessun prerequisito sui dati"]
    A --> B["Ricerca Binaria (Dicotomica)\nComplessità: O(log n)\nPrerequisito: Dati

    style L fill:#fff3cd,stroke:#ffc107,color:#856404
    style B fill:#d4edda,stroke:#28a745,color:#155724
```

A. Ricerca Lineare (Sequenziale)

Scorre gli elementi uno dopo l'altro dall'inizio alla fine.
In Python possiamo scriverla manualmente oppure affidarci all'operatore nativo `in`:

```
# Modo manuale:
def ricerca_lineare(lista: list, bersaglio) -> int:
    for i, valore in enumerate(lista):
        if valore == bersaglio:
            return i  # Trovato: restituisce l'indice
    return -1         # Non trovato

# Modo nativo pythonico:
numeri = [40, 10, 85, 90, 12]
if 85 in numeri:
    indice = numeri.index(85)
    print(f"85 trovato all'indice {indice}")
```

B. Ricerca Binaria (Dicotomica)

Funziona solo su liste già ordinate: confronta l'elemento centrale e dimezza lo spazio di ricerca a ogni passaggio, riducendo le operazioni a O(log2 n).

```
def ricerca_binaria(lista: list, bersaglio: int) -> int:
    basso = 0
    alto = len(lista) - 1

    while basso <= alto:
        medio = (basso + alto) // 2
        if lista[medio] == bersaglio:
            return medio
        elif lista[medio] < bersaglio:
            basso = medio + 1
        else:
            alto = medio - 1

    return -1
```

> 💡 **Il Modulo Nativo `bisect`**

> Python include il modulo `bisect` compilato in C per la ricerca e l'inserimento ordinato istantaneo:
>
> ```
> import bisect
> lista_ordinata = [10, 20, 30, 40, 50]
> pos = bisect.bisect_left(lista_ordinata, 30)  # Restituisce 2
> ```

---

2. Gli Algoritmi Didattici di Ordinamento

Anche se nella pratica si usano sempre gli strumenti nativi, l'implementazione degli algoritmi classici è fondamentale per comprendere la complessità computazionale:

A. Bubble Sort Ottimizzato

Sfrutta la caratteristica eleganza di Python: lo **scambio multiplo di variabili in una riga** (`a, b = b, a`) che non richiede variabili ausiliarie `temp` come in C++:

```
def bubble_sort(arr: list) -> list:
    n = len(arr)
    for i in range(n):
        scambiato = False
        for j in range(0, n - i - 1):
            if arr[j] > arr[j + 1]:
                # Scambio pythonico istantaneo
                arr[j], arr[j + 1] = arr[j + 1], arr[j]
                scambiato = True
        if not scambiato:
            break  # Ottimizzazione: lista già ordinata!
    return arr
```

B. Selection Sort (Ordinamento per Selezione)

Cerca progressivamente il valore minimo e lo scambia con l'elemento di testa:

```python
def selection_sort(arr: list) -> list:
    n = len(arr)
    for i in range(n):
        min_idx = i
        for j in range(i + 1, n):
            if arr[j] < arr[min_idx]:
                min_idx = j
        arr[i], arr[min_idx] = arr[min_idx], arr[i]
    return arr
```

---

3. L'Algoritmo di Sistema: Timsort

Quando chiami un ordinamento in Python, non stai eseguendo un semplice QuickSort o MergeSort, ma **Timsort**, un algoritmo ibrido ad altissima efficienza ideato nel 2002 da **Tim Peters**.

```mermaid
flowchart LR
    A["Dati in Ingresso"] --> B["Individuazione di sequenze già ordinate\n('Natural
    B --> C["Ordinamento brevi blocchi con Insertion Sort"]
    C --> D["Fusione gerarchica dei blocchi con Merge Sort"]
    D --> E["Lista Perfettamente Ordinata"]

    style A fill:#e1f5fe,stroke:#03a9f4,color:#01579b
    style B fill:#fff9c4,stroke:#fbc02d,color:#f57f17
    style C fill:#ffe0b2,stroke:#fb8c00,color:#e65100
    style D fill:#c8e6c9,stroke:#4caf50,color:#1b5e20
    style E fill:#d4edda,stroke:#28a745,color:#155724
```

Perché Timsort è Rivoluzionario?

1. **Complessità nel Caso Migliore**: $O(n)$ quando i dati sono già (o quasi) ordinati, invece di $O(n \log n)$.
2. **Caso Peggiore e Medio**: $O(n \log n)$ garantito matematicamente (nessun caso degenere $O(n2)$ come il QuickSort).

3. **Stabilità**: è un algoritmo **stabile** (preserva l'ordine relativo originale degli elementi aventi chiavi identiche).

Algoritmo	Caso Migliore	Caso Medio	Caso Peggiore	Memoria Extra	Stabile?
Bubble Sort	O(n)	O(n2)	O(n2)	O(1)	Sì
Selection Sort	O(n2)	O(n2)	O(n2)	O(1)	No
Quick Sort	O(n log n)	O(n log n)	O(n2)	O(log n)	No
Timsort (Python)	**O(n)**	**O(n log n)**	**O(n log n)**	O(n)	**Sì**

---

4. `sort()` vs `sorted()`: Non Confonderli!

```
numeri = [5, 2, 9, 1, 7]

# 1. METODO .sort() (In-place: muta l'oggetto originale)
risultato = numeri.sort()
print(numeri)     # [1, 2, 5, 7, 9] (La lista originale è cambiata!)
print(risultato)  # None (NON restituisce una lista, restituisce None!)

# 2. FUNZIONE sorted() (Out-of-place: genera una NUOVA lista)
originale = [5, 2, 9, 1, 7]
nuova_ordinata = sorted(originale)
print(originale)       # [5, 2, 9, 1, 7] (L'originale è INVARIATO!)
print(nuova_ordinata)  # [1, 2, 5, 7, 9]
```

> 🚫 **Errore da Principiante**

> Scrivere `mia_lista = mia_lista.sort()` **cancella la tua lista** sostituendola con `None`! Usa `.sort()` solo come istruzione autonoma.

---

5. Ordinamento Personalizzato con `key` e Funzioni Lambda

Il parametro `key` accetta una funzione (spesso una lambda) che calcola il criterio di comparazione per ogni elemento:

```
studenti = [
    {"nome": "Luca", "eta": 19, "media": 7.5},
    {"nome": "Anna", "eta": 18, "media": 9.2},
    {"nome": "Marco", "eta": 21, "media": 6.8}
]

# 1. Ordina per media decrescente (dal più bravo al meno bravo)
classifica = sorted(studenti, key=lambda s: s["media"], reverse=True)
for pos, s in enumerate(classifica, start=1):
    print(f"{pos}. {s['nome']} - Media: {s['media']}")

# 2. Ordinamento multi-livello (prima per età crescente, poi per media dec
# Sfruttiamo una tupla come chiave di ordinamento!
multi_ordinati = sorted(studenti, key=lambda s: (s["eta"], -s["media"]))
```

---

6. Domande d'Esame e Concetti Chiave

> ❓ **Domande di Verifica**
>
> 1. *Qual è la differenza fondamentale tra il metodo `.sort()` e la funzione `sorted()`?*
>   **Risposta**: `.sort()` modifica direttamente sul posto la lista originale restituendo `None`. La funzione `sorted()` non altera la collezione di partenza e restituisce una nuova lista con gli elementi ordinati.

2. *Qual è il prerequisito indispensabile per poter eseguire una Ricerca Binaria?*
   Risposta: La sequenza di elementi deve essere già rigorosamente ordinata prima di avviare l'algoritmo.
3. *Cosa significa che l'algoritmo Timsort è "stabile"?*
   Risposta: Significa che se due elementi hanno lo stesso valore di chiave, il loro ordine relativo iniziale nella sequenza originale viene preservato esattamente anche dopo l'ordinamento.

---

7. Programma Esecutivo Completo

🧪 **Programma Completo: Benchmark di Confronto Prestazionale tra Algoritmi**

```
import random
import time

def benchmark_ordinamento():
    # Generiamo una lista di 2000 numeri casuali
    dimensione = 2000
    dati_casuali = [random.randint(1, 10000) for _ in range(dimension

    print(f"=== BENCHMARK ORDINAMENTO SU {dimensione} ELEMENTI ===")

    # 1. Test Bubble Sort
    copia_bubble = dati_casuali.copy()
    t0 = time.perf_counter()
    # Implementazione Bubble Sort
    n = len(copia_bubble)
    for i in range(n):
        scambio = False
        for j in range(0, n - i - 1):
            if copia_bubble[j] > copia_bubble[j + 1]:
                copia_bubble[j], copia_bubble[j + 1] = copia_bubble[j
                scambio = True
        if not scambio:
            break
    t_bubble = time.perf_counter() - t0
    print(f"1. Bubble Sort (O(n²)) : {t_bubble:.4f} secondi")

    # 2. Test Timsort nativo di Python (C-speed)
    copia_timsort = dati_casuali.copy()
    t0 = time.perf_counter()
    copia_timsort.sort()
    t_timsort = time.perf_counter() - t0
    print(f"2. Python Timsort (O(n log n)): {t_timsort:.6f} secondi")

    fattore = t_bubble / t_timsort if t_timsort > 0 else 0
    print("-" * 45)
    print(f"🚀 Timsort è stato circa {fattore:.1f}x volte più veloce

if __name__ == "__main__":
    benchmark_ordinamento()
```